Fluxo de Dados

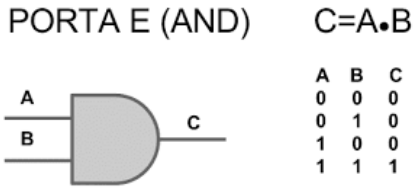
Tuesday, April 21, 2026 2:01 PM

Os fluxos condicionais são a espinha dorsal da lógica de programação em baixo nível, permitindo que o processador tome decisões em tempo real. Para entender como isso funciona na prática dentro da CPU, podemos dividir o processo em três etapas:

1. A Preparação: CMP e TEST

Antes de um salto condicional acontecer, a CPU precisa de uma base de comparação.

- **CMP (Compare):** Subtrai o segundo operando do primeiro, mas **não armazena o resultado**. Ela apenas altera as flags no registrador EFLAGS (como a Zero Flag ou a Sign Flag).
- **TEST:** Realiza uma operação lógica AND, também sem salvar o resultado, sendo muito comum para verificar se um valor é zero ou se um bit específico está ativo.



2. O Tomador de Decisão: Registrador EFLAGS

As instruções de salto não "olham" para os seus números diretamente; elas olham para as **flags** resultantes da operação anterior.

- Se o resultado de um CMP for zero (números iguais), a **ZF (Zero Flag)** é ativada.
- Se o resultado for negativo, a **SF (Sign Flag)** indica que o primeiro número era menor que o segundo.

3. A Ação: Saltos (Jumps)

Com as flags definidas, as instruções de salto alteram o fluxo de execução desviando o **Instruction Pointer (RIP/EIP)**.

Instrução	Descrição	Flag Condição
JMP	Salto Incondicional	Salta sempre, sem checar flags.
JE / JZ	Jump if Equal / Zero	Salta se ZF = 1.
JNE / JNZ	Jump if Not Equal	Salta se ZF = 0.
JG / JL	Jump if Greater / Less	Usado para números com sinal.
JA / JB	Jump if Above / Below	Usado para números sem sinal (unsigned).

Exemplo Prático de Fluxo

if (x == 10):

1. O Assembly executará CMP EAX, 10.
2. Se EAX for 10, a Zero Flag se torna 1.
3. A instrução seguinte, JE label_sucesso, verifica a flag e decide: se for 1, o processador pula para o código dentro do if; se for 0, ele simplesmente segue para a próxima linha (o "else" ou o fim do bloco).

Registrador EFLAGS

Tuesday, April 21, 2026 2:07 PM

- Esse registrador é consultado após comparações e testes para determinar o fluxo de execução, como em saltos condicionais
- Operações como cmp e test ajustam essas flags sem alterar os operandos, alterando apenas decisões subsequentes no código

Bit	Nome	Sigla	Descrição
0	Carry	CF	Ativado quando o resultado estoura o limite do tamanho do dado "Vai um" da matemática para números sem sinal (*unsigned*)
6	Zero	ZF	Ativado quando o resultado de uma operação é zero Utilizada em comparações
7	Sign	SF	Ativada de acordo com o MSB (*Most Significant Bit*) do resultado, que é o *bit* que define se um inteiro com sinal é positivo (0) ou negativo (1)
11	Overflow	OF	Estouro para números com sinal

1. Carry Flag (CF) - O "Vai Um" para Unsigned

- **Cenário:** 0xFFFFFFFF é o valor máximo para 32 bits.
- **O que ocorre:** Ao somar 1, o resultado precisaria de um 33º bit para ser representado. Como o registrador só tem 32, o bit "sobra" e ativa a **CF**.
  +1
- **Uso:** Essencial para aritmética de números sem sinal (unsigned).

2. Overflow Flag (OF) - O Erro de Sinal (Signed)

- **Cenário:** 0x7FFFFFFF é o maior número positivo que cabe em 32 bits (com sinal).
- **O que ocorre:** Ao somar 1, o bit de sinal (o mais significativo) muda para 1, o que transformaria um número positivo em negativo erroneamente. A CPU detecta essa inconsistência lógica e ativa a **OF**.
  +1

3. Sign Flag (SF) - O Bit de Negativo

- **Cenário:** 0x80000000 representa o valor mínimo negativo em 32 bits.
- **O que ocorre:** Subtrair 1 aqui altera o MSB (Most Significant Bit). A **SF** simplesmente reflete o estado desse bit: se o resultado da operação tem o bit de sinal como 1, a flag é ligada.
  +1

4. Zero Flag (ZF) - O Detector de Igualdade

- **Cenário A:** 1 - 1 = 0. Qualquer operação que resulte em zero absoluto ativa a **ZF**.
- **Cenário B (TEST):** A instrução test eax, eax realiza um AND lógico.
- No terceiro exemplo (mov eax, 0), 0 AND 0 = 0, então a **ZF é ativada**.

```
# Carry Flag (CF)
mov eax, 0xFFFFFFFF
add eax, 1          # Ultrapassa o limite de 32 bits, ativando CF

# Overflow Flag (OF)
mov eax, 0x7FFFFFFF  # Máximo positivo em eax
add eax, 1           # Overflow ocorre, dessa vez acontece feijoada

# Sign Flag (SF)
mov eax, 0x80000000  # Valor mínimo negativo em eax
sub eax, 1           # Ativa SF ao se tornar positivo

# Zero Flag (ZF)
mov eax, 1
sub eax, 1           # Ativa ZF

mov eax, 1
test eax, eax       # Como eax é 1, ZF é ativado

mov eax, 0
test eax, eax       # Como eax é 0, ZF é ativado
```

Funções

Tuesday, April 21, 2026 2:23 PM

1. O Controle de Fluxo: CALL e RET

Diferente de um salto comum (JMP), uma função precisa saber de onde veio para poder voltar após terminar a tarefa.

- **CALL:** Esta instrução faz duas coisas: ela salva o endereço da próxima instrução na **pilha (stack)** e depois desvia a execução para o rótulo da função.
- **RET:** É usada no final da função. Ela retira o endereço que foi salvo na pilha e coloca de volta no registrador RIP (Instruction Pointer), fazendo o programa continuar exatamente de onde parou.

2. Calling Convention (Convenção de Chamada)

Para que diferentes partes de um sistema (ou código em C e Assembly) conversem, existe um "contrato" chamado **ABI (Application Binary Interface)**. Ele define quem é responsável por cada registrador:

- **Passagem de Argumentos:** Em sistemas Linux x86-64, os primeiros 6 argumentos são passados obrigatoriamente nesta ordem de registradores: RDI, RSI, RDX, RCX, R8 e R9.
- **Valor de Retorno:** O resultado final da função deve ser sempre colocado no registrador **RAX**.
- **Preservação de Registradores:**
- **Caller-saved (R10, R11):** Se o chamador quiser manter esses valores, ele deve salvá-los antes de chamar a função.
- **Callee-saved (RBX, RBP, R12-R15):** Se a função precisar usar esses registradores, ela é obrigada a salvar o valor original e restaurá-lo antes de dar o RET.

3. Anatomia de uma Função (Prólogo e Epílogo)

Para garantir a organização da pilha e dos registradores, as funções seguem um ritual:

- **Prólogo (Início):**

1. Aloca espaço para variáveis locais (ex: sub rsp, N).
2. Salva os registradores que a função irá modificar e que devem ser preservados (ex: push rbx).

- **Corpo da Função:** Executa a lógica desejada.
- **Epílogo (Fim):**

1. Coloca o resultado em RAX.
2. Restaura os registradores salvos na ordem inversa (pop).
3. Libera o espaço da pilha (add rsp, N).
4. Finaliza com RET.

```
funcao:
    xor eax, eax
    add rax, rdi
    add rax, rsi
    add rax, rdx
    ret

_start:
    mov rdi, 10
    mov rsi, 20
    mov rdx, 30
    call funcao
```

Chamadas de Sistema

Tuesday, April 21, 2026 2:29 PM

Uma chamada de sistema, ou syscall (abreviação para system call), funciona de maneira similar à instrução call mas com a diferença em ser algo onde o comando é passado para o sistema operacional.

1. Por que elas existem? (O conceito de Privilégio)

O hardware do computador opera em diferentes níveis de privilégio. Um programa comum (User Mode) não pode acessar diretamente o disco rígido, a placa de rede ou a memória de outros programas por questões de segurança e estabilidade. Se ele precisa de algo, deve pedir ao **Kernel (Kernel Mode)**, que é o único com poder total sobre o hardware.

2. Como funciona a instrução syscall

Na arquitetura x86-64, existe uma instrução específica chamada syscall. Ela não recebe operandos diretamente no comando, mas segue uma convenção rigorosa:

- **Identificação:** Você coloca o **número da syscall** (que identifica se você quer ler um arquivo, escrever na tela, fechar o programa, etc.) no registrador **RAX**.
- **Argumentos:** Assim como nas funções, os argumentos seguem uma ordem nos registradores: **RDI, RSI, RDX, R10, R8, R9**.
- **A Execução:** Ao chamar syscall, o processador interrompe o programa, muda o nível de privilégio para o Kernel, executa a tarefa e depois devolve o controle ao programa.

3. Exemplo Prático: Finalizando um Programa

Para encerrar um processo de forma limpa no Linux (equivalente ao exit() em C), os passos em Assembly seriam:

1. Colocar o número **60** (que é o ID da syscall exit) em RAX.
2. Colocar o código de retorno (ex: **0** para sucesso) em RDI.
3. Executar a instrução syscall.

4. Diferença entre CALL e SYSCALL

É comum confundir as duas, mas a diferença é crucial:

- **CALL:** Chama uma função dentro do seu próprio programa. Você permanece no mesmo nível de privilégio (**User Mode**).
- **SYSCALL:** Salta para fora do seu programa e entra no código do **Sistema Operacional**. Há uma mudança de privilégio para o **Kernel Mode**.

Finalidade	Registrador
ID da Chamada	**RAX**
Argumento 1	**RDI**
Argumento 2	**RSI**
Argumento 3	**RDX**
Argumento 4	**R10**
Argumento 5	**R8**
Argumento 6	**R9**

```
.intel_syntax noprefix
.global _start
.section .text

_start:
    mov rax, 60
    mov rdi, 42
    syscall
```

- O número **60** é o identificador da syscall exit no Linux x86-64.
- Ao colocar esse valor no **RAX**, você está dizendo ao Sistema Operacional: *"Eu quero fechar este processo"*.
- O registrador **RDI** guarda o primeiro argumento da syscall.
- Syscall: O processador para a execução do seu código, sobe o nível de privilégio para **Kernel Mode** e entrega o controle para o sistema operacional processar o pedido que você montou nos registradores.

```
.section .text
_start:
    mov rax, 1
    mov rdi, 1
    lea rsi, msg
    mov rdx, len
    syscall

    mov eax, 60
    xor edi, edi
    syscall

.section .data
msg:    .asciz "IDP\n"
len:    .byte 0x4
```

1ª Chamada de Sistema: sys_write (Escrever)

Para imprimir algo no terminal, o programa preenche os seguintes requisitos:

- **mov rax, 1**: O número **1** identifica a syscall write no Linux x86-64.
- **mov rdi, 1**: Define o destino da escrita. O número **1** aqui representa o *file descriptor* da **saída padrão (stdout)**.
- **lea rsi, msg**: Carrega o endereço de memória onde a string "IDP\n" está armazenada.
- **mov rdx, len**: Informa ao sistema quantos bytes devem ser lidos da memória (neste caso, 4 bytes).
- **syscall**: Transfere o controle para o Kernel realizar a impressão.

2ª Chamada de Sistema: sys_exit (Sair)

Após imprimir a mensagem, o programa precisa avisar ao Sistema Operacional que terminou:

- **mov eax, 60**: O número **60** identifica a syscall exit. Observe que usar eax automaticamente zera os bits superiores de rax, o que é eficiente.
- **xor edi, edi**: Uma forma rápida de zerar o registrador rdi. O status de saída será **0** (indicando que o programa rodou sem erros).
- **syscall**: O Kernel encerra o processo.

O Bloco .section .data

É aqui que os dados estáticos são definidos:

- **msg: .asciz "IDP\n"**: Reserva espaço para o texto, incluindo o caractere de nova linha (\n).
- **len: .byte 0x4**: Define uma constante com o tamanho da mensagem para ser usada no registrador rdx.

Obs. RAX e RDI

Tuesday, April 21, 2026 2:51 PM

A diferença fundamental entre o **RAX** e o **RDI** não está na sua estrutura física (ambos são registradores de 64 bits para uso geral), mas sim no **papel que eles desempenham** dentro das convenções de chamada do sistema (ABI) e das chamadas de sistema (Syscalls).

1. RAX (O Acumulador e Identificador)

O **RAX** é historicamente o registrador "Acumulador". Suas funções principais são:

- **Identificador de Syscall:** Quando você usa a instrução syscall, o Kernel olha para o **RAX** para saber **o que fazer**. Se houver 60, ele encerra o programa; se houver 1, ele escreve dados.
- **Valor de Retorno:** Quando uma função termina, o resultado final (como o return x em C) é quase sempre colocado no **RAX** para que quem chamou a função possa ler o resultado.
- **Aritmética:** É o registrador preferencial para operações de soma, multiplicação e divisão.

2. RDI (O Destino e Primeiro Argumento)

O **RDI** é historicamente o "Destination Index". Suas funções principais são:

- **Primeiro Argumento:** De acordo com a convenção de chamadas do Linux x86-64, o **primeiro argumento** que você passa para uma função ou syscall deve estar no **RDI**.
- **Dados para a Syscall:** Na syscall exit (ID 60), o **RDI** guarda o código de status. Na syscall write (ID 1), o **RDI** guarda onde você quer escrever (como o monitor/stdout).
- **Operações de String:** É usado para apontar para o endereço de destino em operações que movem blocos de memória.

Resumo prático

Ordem para a CPU:

- O **RAX** é o **código da tarefa** no manual de instruções.
- O **RDI** é a **primeira informação** necessária para realizar aquela tarefa específica.